SALEM–SPENCER SETS AS A CROSS-SOLVER BENCHMARK: DECOMPOSITION, CERTIFICATION, AND SPLIT-POLICY EFFECTS AT $r_3(212)$

MEHMET ERGEZER

Abstract. Deciding whether a 44-element subset of $[1, 212]$ can avoid all three-term arithmetic progressions is the current frontier instance of the Salem–Spencer sequence $r_3(N)$ (OEIS A003002). We present this family of finite feasibility problems as a cross-solver benchmark with a measured difficulty ladder. None of six monolithic CP-SAT, HiGHS, and CDCL configurations resolves the instance within 24 hours. A witness-informed cube-and-conquer campaign closes millions of subproblems with no feasible 44-set, supporting but not proving $r_3(212) = 43$; OEIS-derived window-cardinality inequalities reduce the controlled broad-pass unresolved rate by about 28 percentage points. A post-campaign implementation audit and matched five-policy ablation identify a global AP-degree prefix with 96,847 surviving cubes, compared with 12,582,912 for the deployed witness prefix, while showing that the rarer survivors require a stronger conquer phase. On byte-identical hard-pocket formulas, native CaDiCaL 3.0.0 and kissat 4.0.4 both close all 20 tested chunks; the two historically hardest kissat refutations are independently DRAT-verified. A full two-stage CDCL survey closes 6,045/6,071 broad residuals (99.57%) without proof logging. Optimization-form MIP experiments show that window inequalities tighten all 20 tested LP and MIP upper bounds to 44 but do not cross the decision threshold. As an end-to-end ground-truth check, the pure encoding recovers $r_3(80) = 22$, $r_3(90) = 24$, and $r_3(100) = 27$ from independently verified lower witnesses and LRAT certificates accepted by the formally verified `cake_lpr` checker. The unit gap $r_3(212) \in \{43, 44\}$ remains open. We release the tiered instances, deterministic generators, verified certificates, model audit, and formal-proof-search targets.

1. Introduction and background

Let $r_3(N)$ denote the maximum size of a subset of $[1, N]$ containing no nontrivial three-term arithmetic progression. Equivalently,

$$r_3(N) \;=\; \max\{\, |A| \;:\; A \subseteq [1, N],\ \text{no } a < b < c \text{ in } A \text{ satisfy } a + c = 2b \,\}.$$

The asymptotic study of $r_3(N)$ begins with Roth's theorem [Rot53] and continues through Salem–Spencer and Behrend-type lower-bound constructions [SS42, Beh46] and modern density-increment upper bounds, including work of Bloom–Sisask and Kelley–Meka [BS20, KM23]. Those results describe the large-N behavior of progression-free sets. The present paper is about a different but complementary problem: exact finite computation at the OEIS A003002 frontier.

OEIS A003002 tabulates exact values of $r_3(N)$. Cariboni's b-file currently reaches $N = 211$, where $r_3(211) = 43$ [OEI24, Car24]. The next natural decision problem is therefore:

Does there exist a 44-element 3-AP-free subset of $[1, 212]$?

Date: July 15, 2026.

2020 *Mathematics Subject Classification.* Primary 11B25; Secondary 11Y16, 68W30, 68V20, 90C10 .

Key words and phrases. constraint programming, benchmark instances, Salem–Spencer numbers, arithmetic progressions, OEIS A003002, CP-SAT, MIP, CDCL, DRAT proofs, formal verification, Lean .

ORCID: 0000-0001-6627-3667.

We found and independently verified a 43-element 3-AP-free subset of $[1, 212]$, so the lower bound $r_3(212) \geq 43$ is settled. The upper-bound question is whether $r_3(212) \leq 43$. Since $r_3(211) = 43$, any hypothetical 44-set in $[1, 212]$ must contain both endpoints 1 and 212; otherwise it would translate or restrict to a 44-set in $[1, 211]$. Thus the target is a single finite feasibility problem with a strong necessary endpoint condition.

Finite feasibility problems of this Ramsey-combinatorial flavor have been a showcase for modern SAT and constraint solving: the Boolean Pythagorean triples problem [HKM16], Schur number five [Heu18], and Keller's conjecture in dimension seven [BHMN22] were each closed by massively parallel CDCL search with machine-checkable proofs. The Salem–Spencer family sits in the same problem class — Boolean variables, one short linear constraint per arithmetic progression, a cardinality target — and, as this paper documents, it separates tested solver configurations unusually sharply. None of six monolithic CP-SAT, HiGHS, or CDCL configurations resolves the frontier instance in 24 hours. Under a witness-informed split, CP-SAT and HiGHS leave a common residual pocket, while CDCL closes it. An initial PySAT-bundled CaDiCaL configuration left two audited chunks unresolved; kissat subsequently refuted both with independently verified DRAT proofs. A same-formula rerun on byte-identical CNFs then showed that current native CaDiCaL 3.0.0 and kissat 4.0.4 both close all 20 hard chunks. The observed deployment times differ, but the first array used heterogeneous processors; §4.4 reports that limitation separately from the solvability result. That layered behavior is precisely what makes the family interesting as a benchmark: it is natural, scalable in N, has verifiable ground truth at solved sizes, and grades solver configurations along a measured difficulty ladder rather than a single pass/fail line.

We attacked this feasibility problem computationally using a reproducible CP-SAT architecture. The model uses Boolean variables x_i, one linear constraint for each 3-term arithmetic progression, the decision equality $\sum_i x_i = 44$, endpoint forcing, reflection symmetry breaking, and window-cardinality inequalities derived from the known values of $r_3(L)$ for $L \leq 211$. To make the search tractable, we split the problem by assigning a fixed prefix drawn from a verified 43-witness, generating a deterministic depth-24 chunk space, and then refine residual `UNKNOWN` chunks recursively.

The campaign did not produce a formal proof of $r_3(212) = 43$. It did produce six concrete outcomes.

First, we observed zero feasible 44-sets across millions of CP-SAT subproblems, including broad chunks, recursive refinements, wall-cap recalibrations, and targeted A/B experiments. This is empirical evidence for the expected value $r_3(212) = 43$, but it is not a certificate.

Second, we found that OEIS window-cardinality inequalities are essential. On controlled broad-pass ranges, adding these inequalities reduced the `UNKNOWN` rate by roughly 28 percentage points and also reduced aggregate solver time. They are the main successful pruning layer of the campaign.

Third, we identified and stratified a structural hard pocket. The largest retained broad run over 100,000 depth-24 chunks left 6,071 `UNKNOWN` chunks. A uniform random 100-chunk sample of those 6,071 (seed 5230) was passed through a 300-second recap, leaving 45 resistant survivors. Re-attacking those 45 chunks with HiGHS, an LP-relaxation-based MIP solver, closed 0/45 in one-hour-per-chunk runs. A full eight-hour audit later closed 25/45, but left 20/45 `UNKNOWN`. A subsequent pure CDCL/SAT attack (CaDiCaL via PySAT) closed 18/20 of that CP-SAT/HiGHS-resistant subset, while two chunks — T1c — remained `UNKNOWN` even under a 12-hour pure-CDCL cap and a 4-hour windowed-CDCL diagnostic in that initial configuration.

Fourth, the apparent pocket collapses under current native CDCL solvers. Kissat 4.0.4 refutes both T1c chunks on the pure 3-AP/cardinality encoding, and both refutations are certified end-to-end against sha256-pinned formula files (§4.4). In a same-formula 20-chunk comparison on byte-identical CNFs, native CaDiCaL 3.0.0 and kissat both close every chunk; kissat is faster on

19/20 heterogeneous-node deployment runs, a timing observation that does not isolate solver speed. A full kissat pass over all 6,071 broad residuals closes 5,959 within a 2-hour cap, and a 12-hour native-CaDiCaL pass closes 86 of the 112 survivors. The resulting two-solver portfolio leaves 26 unresolved chunks and no feasible row, for 99.57% measured coverage (§3.8).

Fifth, we validate the complete encoding and certificate chain at solved sizes. Pure 3-AP/cardinality instances recover $r_3(80) = 22$, $r_3(90) = 24$, and $r_3(100) = 27$: each lower witness passes an independent combinatorial check, and each upper refutation is converted to LRAT and accepted by the formally verified `cake_lpr` checker (§3.9).

Sixth, a post-campaign implementation audit corrected the record of the deployed split policy and enabled a matched five-policy ablation. Exact prefix enumeration reduces the complete depth-24 cover from 12,582,912 surviving cubes under the deployed witness-numeric prefix to 96,847 under global AP-degree. On 500 matched raw assignments, global degree improves the 60-s closed rate by 13.4 percentage points, while all three sampled assignments that survive its prefix pruning remain `UNKNOWN`. The experiment therefore exposes a genuine cover-size versus residual-hardness tradeoff rather than a simple speedup (§3.7).

This paper's contribution is the benchmark and the cross-solver empirical study, not a new exact value of A003002. We provide:

1. A tiered benchmark release spanning a measured difficulty ladder: the 25 HiGHS-closable T1a chunks, the 18 chunks closed by the initial CDCL configuration, the 2 historically hardest T1c chunks, the 6,071 broad `UNKNOWN` chunks from the 100k expansion batch, and a deterministic generator for the full depth-24 sweep.
2. A characterization of the structural hard pocket across CP-SAT, HiGHS, and multiple CDCL configurations. The pocket that initially appeared robust across formulations is closed by both native CaDiCaL 3.0.0 and kissat 4.0.4; the initial timing data are reported as a heterogeneous-node deployment comparison rather than a processor-normalized speedup.
3. A reproducible witness-split plus window-cardinality architecture for exact $r_3(N)$ upper-bound search, parametric in (N, K) and reusable at adjacent frontier sizes. The problem-specific window-cardinality inequalities are, empirically, worth more than any generic solver lever we tested.
4. A hardware-matched, five-policy split ablation with exact prefix-survivor counts. It documents the deployed loader behavior and identifies a global-degree cover with 96,847 cubes, 130 times fewer than the historical witness-anchored cover, while measuring the increased hardness of the surviving cubes.
5. Three independently checked exact-value regressions at $N = 80, 90, 100$, including verified lower witnesses, archived pure CNFs, DRAT-to-LRAT conversion, and formal certificate checking with `cake_lpr`.
6. A detailed empirical account of the $N = 212$, $K = 44$ campaign, including broad-pass counts, refinement behavior, solver-time tradeoffs, controlled A/B experiments, and an implementation audit that identifies an intended split-reordering run as a no-op.

For reference, the benchmark tier notation used throughout the paper is:

Symbol	Meaning
T1	the 45 chunks that survived the 300-s recap of a random 100-chunk sample from the 6,071 broad UNKNOWNs
T1a	the 25 T1 chunks closed by the full 8-h HiGHS audit
T1b	the 20 T1 chunks left UNKNOWN by the full 8-h HiGHS audit

Symbol	Meaning
T1b \ T1c	the 18 T1b chunks closed by the initial CDCL configuration and independently verified by DRAT checking
T1c	chunks $\{40959, 48895\}$: resisted the initial CP-SAT, HiGHS, and PySAT-bundled CaDiCaL configurations; both native CaDiCaL 3.0.0 and kissat 4.0.4 refute them
T2	the 6,071 UNKNOWN chunks from the 100k broad expansion batch
T3	a complete AP-pruned depth-24 cover: 12,582,912 cubes under the deployed witness prefix, or 96,847 under global AP-degree

1.1. **Related work. Split-and-solve search.** The witness-split architecture of §2.2 belongs to the cube-and-conquer family [HKWB12]: partition the search space by fixing a prefix of variable assignments (cubes), then solve each cube with a complete solver. The differences are the cube-selection policy and the refinement loop. Classical cube-and-conquer selects split variables by lookahead heuristics; we restrict the split variables to a verified extremal witness and use a fixed, explicitly released witness prefix. This exploits the problem-specific intuition that a hypothetical 44-set must interact heavily with a known 43-set. Timed-out cubes are then re-split recursively (§2.4), which parallels iterative cube deepening.

Implied constraints and streamliners. The window-cardinality inequalities of §2.3 are implied constraints in the standard CP-modeling sense: they are logical consequences of the base 3-AP model that are not explicitly present and are not efficiently recovered by the tested propagators. They are derived from previously *computed* values of the same sequence — the solved prefix of A003002 prunes the frontier instance. This is soundness-preserving, in contrast to streamlined constraint reasoning [GS04], which deliberately sacrifices completeness; it is closer in spirit to bound propagation from tabulated subproblem optima. Empirically these inequalities dominate every generic lever we tested (§3.3, §4.5).

SAT and CP in combinatorial mathematics. Beyond the celebrated results cited above, exact computation of Salem–Spencer values has its own lineage: Dybizbański [Dyb12] computed $r_3(N)$ through $N = 123$ with a tailored branch-and-bound, and Cariboni's OEIS b-file [Car24] extends the sequence to $N = 211$. Our campaign differs in attacking the frontier with general-purpose solvers plus problem-specific pruning, and in reporting the negative space — which subproblems resist and why — rather than only the closed values.

Benchmarks and certified solving. Curated instance families — CSPLib [GW99], MIPLIB [GHG+21], the SAT Competition suites — have repeatedly steered solver research toward documented failure modes. The tiered release of §5 is offered in that tradition: a natural family, scalable in N, with verifiable ground truth at solved sizes and a measured cross-solver difficulty ladder. On the certification side, the CDCL-closed tier ships with DRAT proofs verified by `drat-trim` [HHB13, CFHH+17]; the absence of comparable certificates for the CP-SAT and MIP closures (§6.2) is an instance of the gap that auditable constraint solving [GMN22] aims to close.

1.2. **Organization.** The rest of the paper is organized as follows. §2 describes the architecture. §3 reports the computational campaign, beginning with the monolithic baseline. §4 analyzes the hard pocket across HiGHS, CaDiCaL, and kissat, ending in its collapse and the refutation of our working hypothesis. §5 releases the tiered benchmark and estimates the remaining path to a full certificate. §6 discusses transferability, limitations, and what the campaign suggests about finite r_3 upper-bound search.

2. Architecture

The campaign rests on five components: a decision-form CP-SAT model of the 3-AP-free subset problem (§2.1), a witness-variable splitting scheme that produces tractable subproblems (§2.2), a window-cardinality pruning family derived from OEIS A003002 (§2.3), a recursive refinement loop for residual `UNKNOWN` chunks (§2.4), and the SLURM-side engineering that makes the workload reproducible (§2.5). Each component is independent of the others — any could be replaced by a stronger variant without disturbing the rest — and together they define the proof attempt on $r_3(212) \leq 43$.

2.1. **Decision-form CP-SAT model.** Fix $N \geq 3$ and $K \geq 3$. The standard CP-SAT encoding of "does there exist a 3-AP-free subset $A \subseteq [1, N]$ with $|A| \geq K$?" introduces Boolean variables $x_i \in \{0, 1\}$ for $i \in [1, N]$, with $x_i = 1$ if and only if $i \in A$, and adds

$$x_a + x_b + x_c \leq 2 \quad \text{for every 3-AP triple } (a, b, c), \qquad \sum_{i=1}^{N} x_i \geq K.$$

We adopt the **decision-form** variant in which the second inequality is replaced by the equality

$$\sum_{i=1}^{N} x_i = K.$$

The decision-form encoding is tighter for propagation, not for logical interpretation of `UNKNOWN`: an `UNKNOWN` return on either encoding is still consistent with both feasibility and infeasibility. The practical advantage of $\sum_i x_i = K$ is that CP-SAT and pseudo-Boolean propagators see both the lower and upper cardinality directions. Branches that would require more than K selected values, or too few remaining unfixed values to reach K, can be cut earlier than in the inequality form. For our purposes (proving the upper bound $r_3(N) \leq K - 1$), the equality form is therefore the right primitive.

For $N = 212$, $K = 44$, the model has 212 Boolean variables and 11,130 3-AP triple inequalities. We add the lex symmetry-breaking constraint

$$(x_1, x_2, \ldots, x_N) \geq_{\text{lex}} (x_N, x_{N-1}, \ldots, x_1).$$

to factor out the reflection $i \mapsto N + 1 - i$. The opposite orientation would be equivalent; the implementation uses the displayed orientation. We do not add other symmetries. After endpoint forcing, reflection is the only order-preserving symmetry we exploit.

We also apply **endpoint forcing** specific to the $r_3(212)$ case. Since OEIS A003002 records $r_3(211) = 43$, any 44-element 3-AP-free subset of $[1, 212]$ must contain both endpoints 1 and 212: if 1 were absent, the set would be a 44-element 3-AP-free subset of $[2, 212]$, which shifts to $[1, 211]$; if 212 were absent, it would already lie in $[1, 211]$. Either case contradicts $r_3(211) = 43$. We add $x_1 = x_{212} = 1$ as ground assumptions in every chunk.

As a check independent of the solver-model builders, the release includes a standard-library-only audit that regenerates the 3-AP triples and window inequalities directly from their mathematical definitions. It checks the monotonic unit-increment structure of the A003002 b-file and records SHA-256 digests of the b-file, the ordered high-level constraint families, and their combined model description. The audit reproduces 11,130 triples and 22,154 window inequalities. This check does not certify a solver's internal CNF or linearization, but it separates errors in the stated combinatorial model from errors in any particular solver adapter.

Branching: variable selection by AP-incidence degree (the number of 3-AP triples a variable appears in), value selection `min`, and `fixed_search` to disable CP-SAT's portfolio search. The first two target the most-constrained variables first; the third reduces solver-policy variation across runs, which is necessary for the A/B experiments of §3.3 and §3.6.

2.2. **Witness-variable splitting.** The decision-form model on the full range $[1, 212]$ is already at the limit of single-call CP-SAT solving in reasonable wall time. To break it into tractable subproblems we use a cube-and-conquer-style split [HKWB12] in which the cubes are chosen not by lookahead but from the verified 43-element lower-bound witness $A_{43} \subset [1, 212]$ (§2.1, §3.2). The splitting proceeds in three steps.

Witness anchoring and deployed prefix. We restrict the split variables to the verified witness A_{43}. The deployed broad run used the first $d = 24$ values after canonical numeric normalization of that witness:

$$\{3, 4, 9, 11, 12, 16, 22, 24, 25, 27, 31, 48, 52, 54, 55, 57, 63, 67, 68, 70, 75, 76, 91, 142\}.$$

We refer to these variables, in the displayed order, as the **broad split prefix**. An implementation audit found that the generic JSON set loader had normalized the stored AP-incidence ordering before truncation. This does not affect coverage or soundness, but it does mean that the deployed cube policy was witness-anchored numeric order rather than the originally intended within-witness degree order. We therefore release the exact prefix and treat split-variable selection as an empirical policy choice, distinct from the AP-degree branching strategy used *inside* each CP-SAT solve.

Combinatorial enumeration. For each of the $2^{24} = 16{,}777{,}216$ raw IN/OUT assignments of the broad split prefix, instantiate a **chunk**: the decision-form model with x_v fixed to the chosen value for every v in the prefix. The 24-bit vector is the raw assignment; after prefix pruning, surviving chunks receive dense integer identifiers in the deterministic selected-first enumeration order.

AP-prefix pruning. Many chunks are immediately infeasible at the 3-AP level alone: if the broad split prefix forces three pinned-IN values that form a 3-AP, the chunk has no feasible completion and need not be sent to CP-SAT. We perform this check during chunk emission and skip such chunks. The surviving chunk count for $N = 212$, $K = 44$ at depth 24 is 12,582,912, roughly 75% of the raw 2^{24} count.

The choice $d = 24$ is empirical: larger d produces more chunks but each is easier; smaller d produces fewer chunks but each is harder. At $d = 24$ the per-chunk solver time at the 60-s wall cap has a heavy tail but a tractable median (the broad pass closes approximately 94% of expansion chunks within 60 s — see §3.2), which makes the depth 24 choice a workable broad layer.

2.3. **Window-cardinality pruning from OEIS A003002.** The 3-AP-free subset problem admits a family of implied constraints. These inequalities are logical consequences of the 3-AP model, but are not explicitly present and need not be recovered efficiently by generic propagation. For any window $[a, a + L - 1] \subseteq [1, N]$ and any length L with $r_3(L) < L$, we add

$$\sum_{i=a}^{a+L-1} x_i \leq r_3(L).$$

This is valid because $A \cap [a, a + L - 1]$ is itself a 3-AP-free subset of an L-element interval, hence of size at most $r_3(L)$. Crucially, the right-hand side $r_3(L)$ is *constant* with respect to a, so the family is a single constant per window length, not a learned bound.

We populate the right-hand sides from the OEIS A003002 b-file (Cariboni's tabulation, valid for $L \leq 211$). For $N = 212$, $K = 44$ we generate window constraints for every available length L with

$r_3(L) < \min(L, K)$ and every $a \in \{1, 2, \ldots, 212 - L + 1\}$. The resulting model contains 22,154 window inequalities.

Empirically the window family is the single most impactful CP-SAT intervention in the campaign: the controlled A/Bs of §3.3 show an approximately 28-percentage-point reduction in `UNKNOWN` rate at the 60-s wall cap when window-bounds are enabled. We use it as the default configuration for every solver call beyond the calibration batch.

2.4. **Recursive refinement.** A chunk that returns `UNKNOWN` at the broad layer is refined by extending the broad split prefix with the next m unfixed values in the same canonical witness order and re-emitting the resulting `2^m` subchunks (less AP-prefix pruning) to the solver under a longer wall cap. The depths and per-step parameters used in the campaign are:

Level	Depth	Step size m	Per-chunk wall cap
Broad	24	—	60 s
L1	40	+16	60 s
L2 (tail)	48	+8	60 s
L3 (level3)	56	+8	60 s
L4 (level4)	64	+8	600 s

Each level is fed only the `UNKNOWN` residuals of the prior level, so the work at deeper levels is concentrated on the hard tail. The expected per-level fan-out `2^m` is mitigated by AP-prefix pruning, which becomes more aggressive at deeper levels (more pinned-IN values, more chances for a 3-AP among them). For example, the Sample-500 L1 stream emits 2,076,105 rows from `500 × 2^16` nominal descendants, a `~6%` survival rate after pruning.

Refinement is the campaign's primary tool for closing residual `UNKNOWN` chunks. It reliably closes individual deep chunks (§3.1 reports 6/6 closure at L4). It does not generalize cheaply to the full 6,071-chunk expansion residual because the per-chunk cost grows roughly geometrically with depth.

2.5. **SLURM engineering.** The campaign runs on the UMass Unity SLURM cluster. The workload pipeline is implemented as a small collection of Python scripts, each with a single responsibility:

- **`r3_slurm_emit.py`** — generates the chunk-ID list for a given `(N, K, split-vars, depth, range)` and emits a SLURM array `sbatch` script that fans out the chunks across array tasks. The `--chunks-per-task` flag controls the granularity of fan-out.
- **`r3_split_cpsat.py`** — the per-task driver. Reads its slice of chunk IDs, instantiates the CP-SAT model for each, runs the solver under the per-chunk wall cap, and appends one JSONL row per chunk to a shard file.
- **`r3_collect.py`** — merges shard files into the canonical batch output. Handles deduplication if a chunk was solved more than once (e.g., during retries).
- **`r3_tail_emit.py`** — emits the next-level refinement workload from the `UNKNOWN` rows of a parent batch. Supports multi-parent inputs (each input JSONL contributing its own parent prefix) and templated output naming via a parent-tag regex.
- **`r3_proof_manager.py`** — tracks the proof-state graph across levels, identifying which chunks have been closed at which depth and which remain open.
- **`r3_verify.py`** — independent triple-enumeration verifier for any candidate K-element witness.

Two engineering decisions are worth highlighting because they affect reproducibility:

Atomic shard renames. Each per-task driver writes its shard to a temporary `.tmp` path keyed by the SLURM job and array IDs, and renames it to `shard.jsonl` on successful completion. A killed or pre-empted task leaves only the temporary file, which is ignored by the collector. This pattern prevents partial output from corrupting downstream collection without requiring any locking.

Deterministic inputs. Every chunk ID maps to a fixed prefix assignment, and every CP-SAT call uses the same solver seed and fixed-search branching. This is necessary for the lever A/B experiments of §3.5: a "no measurable effect" claim is only defensible if the baseline arm is reproducible under the same software stack.

The end-to-end pipeline is driven by `unity_handoff.sh`, which runs the broad pass, the recap studies, the refinement levels, and the HiGHS attack as separately resumable phases. A complete campaign reproduction on a fresh allocation requires the OEIS b-file, the verified 43-witness, the Python environment (OR-Tools, `highspy`, PySAT/CaDiCaL, and `drat-trim`), and a SLURM allocation; everything else is generated by the scripts.

3. Empirical campaign

This section reports the workload, results, and key A/B tests of the $r_3(212) \leq 43$ campaign. All experiments were run on the UMass Unity SLURM cluster against the architecture described in §2. The headline numbers: millions of CP-SAT subproblems solved, 0 `FEASIBLE` rows, and a 6.07% `UNKNOWN` residual in the largest retained broad batch that motivates the structural analysis in §4.

The evaluation is organized around five research questions. **RQ1:** Do general-purpose solvers resolve the monolithic decision instance under a generous wall cap? **RQ2:** Which sound formulation and decomposition choices materially reduce the unresolved rate? **RQ3:** How do CP-SAT, MIP, and native CDCL configurations stratify on byte-identical residual instances? **RQ4:** Does the complete encoding and certificate chain recover independently known exact values? **RQ5:** How much of the frontier residual can a measured solver portfolio close, and what remains for a full certificate? Sections 3.1–3.9 answer these questions with monolithic controls, paired A/B tests, exact prefix counts, same-formula comparisons, formally checked regressions, and a complete T2 survey. Section 4 analyzes the resulting difficulty ladder rather than introducing additional search degrees of freedom after observing the outcomes.

3.1. **Monolithic baseline.** Before any splitting, we hand the full unsplit decision instance — 212 Boolean variables, 11,130 3-AP inequalities, $\sum_i x_i = 44$, endpoint forcing, with and without the 22,154 window-cardinality inequalities — to each tested solver configuration with a 24-hour wall cap.

Tested solver configuration	Windows	Wall	Status	Progress at cap
CP-SAT (8 workers)	on	24 h	`UNKNOWN`	50,127,252 conflicts
CP-SAT (8 workers)	off	24 h	`UNKNOWN`	814,459,413 conflicts
HiGHS feasibility MIP (8 threads)	on	24 h	`UNKNOWN`	837,301 nodes
HiGHS feasibility MIP (8 threads)	off	24 h	`UNKNOWN`	6,800,276 nodes
CDCL (1 core)	on	24 h	`UNKNOWN`	—
CDCL (1 core)	off	24 h	`UNKNOWN`	—

All six cells return `UNKNOWN` at the cap. The two HiGHS runs explore hundreds of thousands to millions of MIP nodes without proving infeasibility. These were zero-objective feasibility models, so their recorded objective and dual bounds of 0.0 are expected and are not evidence about the strength of the LP relaxation. CP-SAT burns through roughly 8×10^8 conflicts on the bare encoding without a refutation. (Conflict counts for the two CDCL cells were not retained by the PySAT

wrapper; both cells ran to the full cap without a result.) None of the tested configurations resolves the monolithic instance, which is the empirical justification for the witness-split architecture of §2.2: the campaign's progress comes from the split and the window bounds, not from raw solver time on the whole problem. (The windowed CDCL cell required 64 GB at encode time; the window family is expensive in CNF form.)

3.2. **Chunk-count breakdown.** The campaign proceeded in three layers of increasing scope, plus a recursive-refinement loop on the `UNKNOWN` residuals.

Verified lower bound. A 43-element 3-AP-free subset of $[1, 212]$ is recorded in the repository's witness JSON and re-verified by `r3_verify.py` against an independent triple-enumeration check. This witness fixes the campaign's $K = 44$ decision-mode target and supplies the seed for the depth-24 broad split (§2.2).

Broad pass at depth 24, 60-s wall. The retained broad-pass logs include both pre-window and window-bound runs:

Range	Chunks	INFEASIBLE	UNKNOWN	UNK rate
Calibration, no window bounds, $[575, 1575)$	1,000	799	201	20.10%
Bounded, no window bounds, $[1575, 11575)$	10,000	6,967	3,033	30.33%
Bounded, with window bounds, $[1575, 11575)$	10,000	9,835	165	1.65%
Expansion, with window bounds, $[11575, 111575)$	100,000	93,929	6,071	6.07%

The 20.10% calibration rate was run *before* window-bounds were enabled and is included for reference only; once window-cardinality pruning is on (see §3.2), the residual sits in the 1–6% band depending on the chunk-ID range.

Recursive refinement of `UNKNOWN` chunks. Every chunk timing out at the broad layer can be refined by extending the witness-pin prefix with the next unfixed values in the canonical witness order and re-solving the resulting subchunks. The number of emitted rows is not a raw `2^d` fan-out: AP-prefix pruning eliminates many descendants before a solver call is made. The retained refinement diagnostics are shown below. The depth labels follow the L1/L2/L3/L4 refinement ladder of §2.4.

Stream	Source	Depth added	Rows emitted	INFEASIBLE	UNKNOWN
Sample-100 L1	random sample of 100 broad UNKs	+16 (to depth 40)	299,375	299,374	1
Sample-100 tail	one Sample-100 residual	+8 (to depth 48)	8	8	0
Sample-500 L1	stratified sample of 500 broad UNKs	+16 (to depth 40)	2,076,105	2,076,095	10
Sample-500 tail8	10 Sample-500 residuals	+8 (to depth 48)	76	68	8
Sample-500 level3	8 tail8 residuals	+8 (to depth 56)	8	2	6
Sample-500 level4	6 level3 residuals at 600-s wall	+8 (to depth 64)	6	6	0

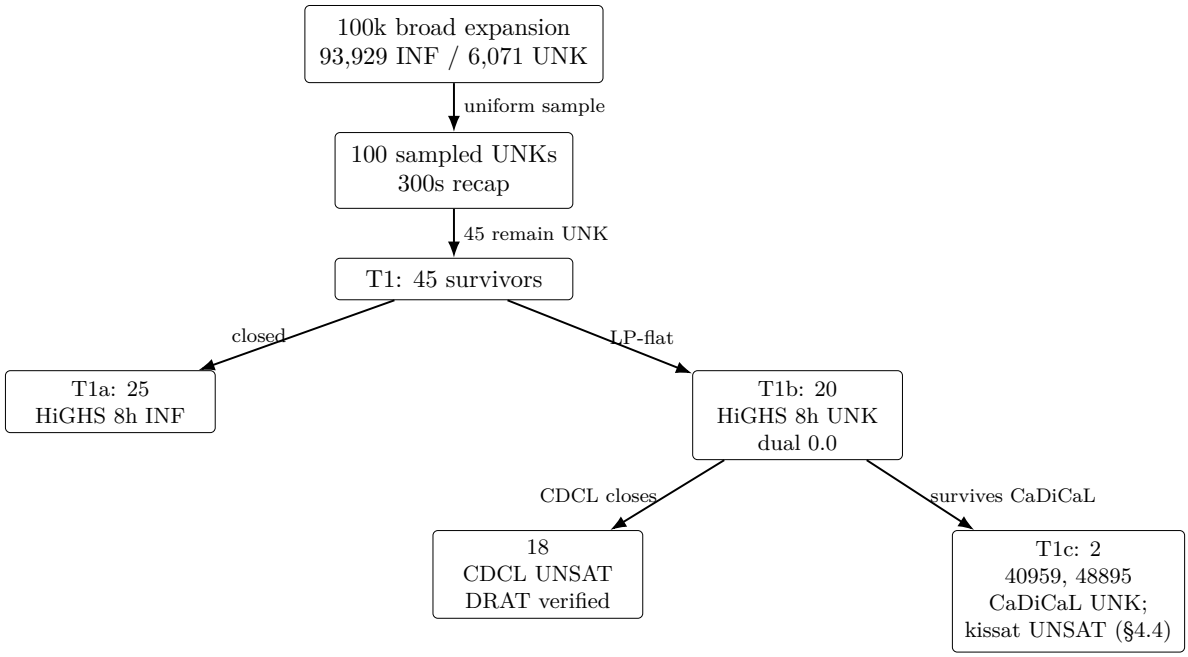

FIGURE 1. Funnel from the retained 100k broad expansion batch to the final two-chunk T1c residual. T1 is a 45-chunk subset produced by recapping a uniform random 100-chunk sample of the 6,071 broad UNKNOWN chunks at 300 seconds (sampling seed 5230). T1c, which survives CaDiCaL, is later refuted by kissat 4.0.4 (§4.4).

The final Sample-500 level4 cleanup closed all 6 deep residuals within the 600-s cap, with a worst-case solver time of 599.74 s — within 0.26 s of the cap, so this cleanup is at the practical limit of the refinement strategy at its current parameter settings. Crucially, none of these levels produced a `FEASIBLE` row.

The 6,071 `UNKNOWN` chunks of the expansion batch were *not* fully refined by this campaign. Three subsets of them were sampled for diagnostic experiments: a uniform random 100-chunk recap study (§3.2 and §4.2), a structural-mining analysis (§4.1), and the HiGHS attack of §4.3.

3.3. **The window-cardinality A/B.** The single most impactful intervention in the campaign was adding window-cardinality inequalities derived from the OEIS A003002 b-file. For every window $[a, a+L-1] \subseteq [1, 212]$ and every length L with $r_3(L) < \min(L, K)$, we add $\sum_{i=a}^{a+L-1} x_i \leq r_3(L)$. For $N = 212$, $K = 44$ this generates 22,154 inequalities, on top of the 11,130 3-AP triple inequalities and the symmetry-breaking lex constraint.

We measured the effect on three batches of varying size:

Batch	Chunks	UNK without window-bounds	UNK with window-bounds	Reduction
$[575, 675)$	100	27.00%	0.00%	−27.0 pp
$[1575, 11575)$	10,000	30.33%	1.65%	−28.7 pp
$[11575, 111575)$	100,000	n/a	6.07%	—

Both controlled A/Bs show a roughly 28-percentage-point reduction in `UNKNOWN` rate at the 60-s broad wall cap. Although the window inequalities are logically implied by the complete 3-AP model, they produce a materially stronger formulation for propagation at this problem size. In the retained logs they also lowered total broad-pass solver time, because many formerly hard chunks closed early under the extra bounds.

The $[575, 675)$ A/B is a 100-chunk subrange of the calibration range reported in §3.1; the difference between its 27.00% no-window `UNK` rate and the calibration range's 20.10% reflects the structural non-uniformity of chunk-ID space rather than a change in model configuration.

However, the `UNK` rate is non-monotone in the chunk-ID range: the small controlled A/B on $[575, 675)$ closes to 0%, the 10k bounded batch sits at 1.65%, and the 100k expansion batch sits at 6.07%. This indicates that the chunk-ID space is structurally non-uniform — later dense chunk IDs (which encode different witness-pin assignments under the selected-first enumeration) are harder in a way that window-bounds do not fully compensate for. The bucket analysis in §4.1 quantifies this non-uniformity.

3.4. **Solver configuration and engineering.** All CP-SAT calls use the following configuration unless explicitly overridden in an experiment:

- Model: decision-mode $\sum_i x_i = 44$, 11,130 3-AP linear inequalities, 22,154 window-cardinality inequalities, lex reflection symmetry break, endpoint forcing $x_1 = x_{212} = 1$.
- Branch strategy: variable selection by AP-incidence degree, minimum-value selection, and fixed-search mode to disable CP-SAT's portfolio.
- Solver: OR-Tools CP-SAT [PF24], 8 workers per task, fixed solver seed and fixed search parameters, wall cap as noted (most commonly 60-s for broad, 300-s and 600-s for recap experiments).

The retained Unity environment used Python 3.12, OR-Tools 9.15.6755, HiGHS 1.14.0, and PySAT 1.9.dev4. The no-proof `pysat:auto` path selected the `cadical195` backend; the later native solver was kissat 4.0.4. Exact command lines and environment captures accompany the released logs.

Unity's shared CPU partition is heterogeneous. Large array jobs were not pinned to one processor model, so their wall times characterize the deployed configurations rather than processor-normalized solver performance. Status comparisons, formula hashes, and certificates are unaffected by this caveat; timing comparisons are labeled accordingly.

The SLURM emitter, shard collector, tail emitter, and proof-state manager implement the workload pipeline. Output is written as line-delimited JSON with one record per chunk; shards are written to a temporary path and atomically renamed on completion so that partial output from killed tasks cannot corrupt downstream collection. The full campaign is reproducible end-to-end via the `unity_handoff.sh` driver.

3.5. **Zero FEASIBLE across the campaign.** Aggregating across the monolithic baseline (§3.1), the broad pass, the refinement loop, the recap studies, the lever experiments of §3.6, the HiGHS attack of §4.3, the CDCL/SAT and proof-producing reruns of §4.3, and the kissat re-attack and T2 survey of §4.4, the campaign solved millions of `(N, K, fixed_in, fixed_out)` subproblems, including prefix-closure work not tabulated above. The number of subproblems returning `FEASIBLE` or `SAT` is **zero**.

The `FEASIBLE` count is the only signal that would directly disprove $r_3(212) \leq 43$; its absence does not constitute a proof of the upper bound, but it is strong empirical support, especially given that the campaign forced the endpoints 1 and 212, which any 44-set must contain by the known value $r_3(211) = 43$, and then explored a depth-24 prefix of the verified 43-witness in the processed

chunk ranges. Within the processed expansion range, a 44-element 3-AP-free subset of $[1, 212]$, if one existed, would have to lie in the 6,071 unresolved broad `UNKNOWN` chunks (or in their deeper refinements). Globally, the much larger unprocessed remainder of the full depth-24 sweep remains open as well.

3.6. **Search-tuning levers and their effect sizes.** We evaluated several CP-SAT-side search-tuning levers. The full lever inventory, including the HiGHS substitution and the pair-AND Tseitin experiment, is consolidated in §4.5; here we report only the three CP-SAT-side levers that operate at the broad layer.

Lever	Mechanism	Bucket	Baseline UNK	Treatment UNK
Reorder attempt (implementation no-op)	Intended to move hot witness pins earlier; both JSON lists were normalized to the same numeric prefix	$[61575, 66575)$	13.60%	12.96%
Wall-cap extension 60 s to 300 s	Same model, same prefix, longer per-chunk budget	random 100 from expansion `UNKNOWN`	100.00%	45.00%
Recursive deepening	Refine `UNK` to depth 64 at 600-s cap	6 L3 residuals	100.00%	0.00%

Post-campaign inspection showed that the intended reorder did not occur: the generic JSON set loader sorted both candidate lists before truncation, so both runs used the same 24 variables in the same numeric order. The 0.64-point difference is therefore a repeatability observation under solver and cluster variation, not evidence about split-variable ordering. We retain the row to document the implementation audit and do not count it as a successful A/B.

The wall-cap extension lever moves the residual substantially on the first doubling but exhibits a saturating closure curve under further extension. The 60 s, 120 s, and 300 s series is analyzed in §4.2; in brief, the marginal close-rate per additional second of wall drops sharply past 300-s, which is why the campaign did not pursue 600-s or 1200-s broad-layer reruns on the full 6,071-chunk expansion residual.

The recursive-deepening lever, applied locally to small residual sets, reliably closes individual deep `UNK` chunks (6/6 at the L4 level in §3.1). But the levers it implies — picking the next unfixed witness variables in the deployed canonical order and re-solving up to `2^8` descendant assignments at a longer wall — do not generalize cheaply to the full 6,071-chunk expansion residual. A naïve depth-32 rerun on every expansion `UNK` would emit roughly 1.5 million subproblems and require an order of magnitude more CPU time than the original broad pass, with no guarantee of closing the hard core analyzed in §4.

3.7. **Matched split-policy ablation.** The implementation audit of §2.2 made split-variable selection an explicit experimental factor. We compared five depth-24 policies: the deployed witness-numeric prefix, the intended within-witness AP-degree prefix, a fixed-seed random witness prefix, global AP-degree, and fixed-seed random global variables. For each policy we first enumerated all raw assignments under endpoint forcing and counted those not eliminated by AP-prefix pruning. The deployed count exactly reproduces the campaign's historical 12,582,912-chunk figure, providing an independent check of the enumeration.

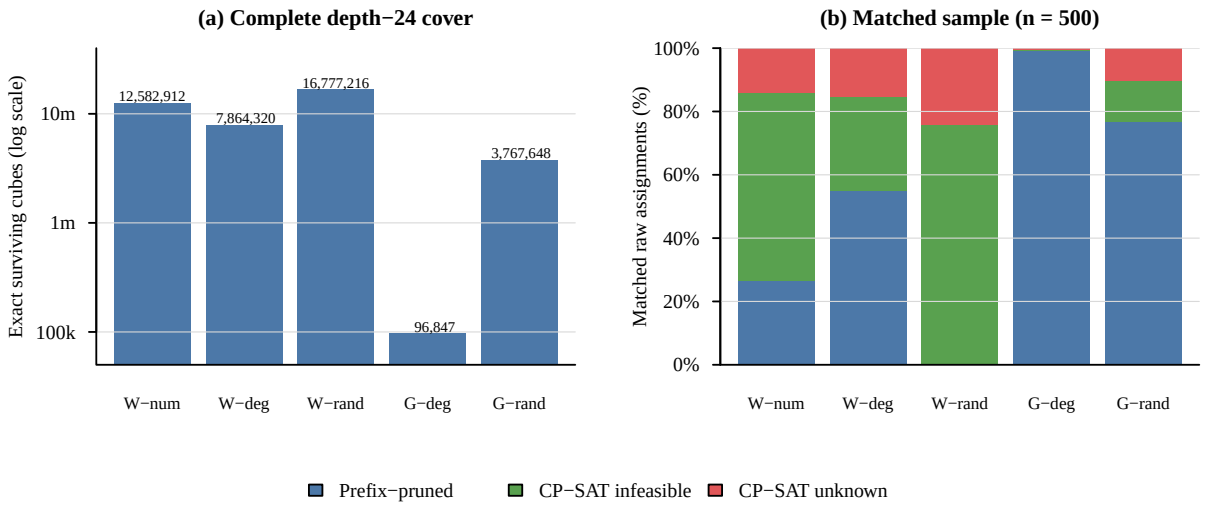

FIGURE 2. Split-policy ablation. Panel (a) gives the exact number of depth-24 cubes that survive AP-prefix pruning. Panel (b) gives outcomes on the same 500 uniformly sampled raw assignments for every policy. Prefix pruning is a sound closure; red segments are unresolved after the 60-s CP-SAT cap. Global degree sharply reduces the complete cover, while its rare survivors require a separate conquer-cost measurement. W and G abbreviate witness-restricted and global policies, respectively.

We then sampled 500 raw 24-bit assignments uniformly with seed 20260715 and applied the same bit vector to every policy. The five arms ran sequentially within each SLURM task on AMD EPYC 7763 nodes; arm order rotated by task index, so every policy occupied each order position 100 times. Unpruned cubes used the campaign CP-SAT configuration (window bounds, degree/min fixed search, eight workers, 60-s cap). No `FEASIBLE` row appeared.

Policy	Exact survivors	Sample pruned	Sample INF	Sample UNK	Mean s/raw
Deployed witness numeric	12,582,912	133	297	70	10.91
Witness degree	7,864,320	275	148	77	12.31
Witness random	16,777,216	0	379	121	19.10
Global degree	96,847	497	0	3	0.36
Global random	3,767,648	383	66	51	7.40

Relative to the deployed policy, global degree increases the matched closed rate by 13.4 percentage points (bootstrap 95% CI 10.4–16.4) and reduces mean solver seconds per raw assignment by 10.55 s (95% CI 8.79–12.37 s). The intended witness-degree policy has no resolved advantage: its closed-rate difference is −1.4 points (95% CI −4.6–2.0). Global random improves the closed rate by 3.8 points (95% CI 0.6–7.0), while witness random is 10.2 points worse (95% CI 7.0–13.4 points worse).

The global-degree result is not a 130-fold solver speedup. Its 132 forbidden prefix masks prune 99.42% of the raw cube space, but all three unpruned cubes encountered in the matched sample reach the 60-s cap. The experiment instead identifies a much smaller complete cover whose residual cubes require a stronger conquer phase. This distinction motivates evaluating cube count and survivor hardness jointly rather than treating prefix-pruning rate alone as the decomposition objective.

3.8. **Full T2 solver portfolio.** The 100-chunk kissat pilot of §4.4 motivated a complete solver survey of all 6,071 T2 chunks. We first ran kissat 4.0.4, single-threaded, on the pure 3-AP/cardinality

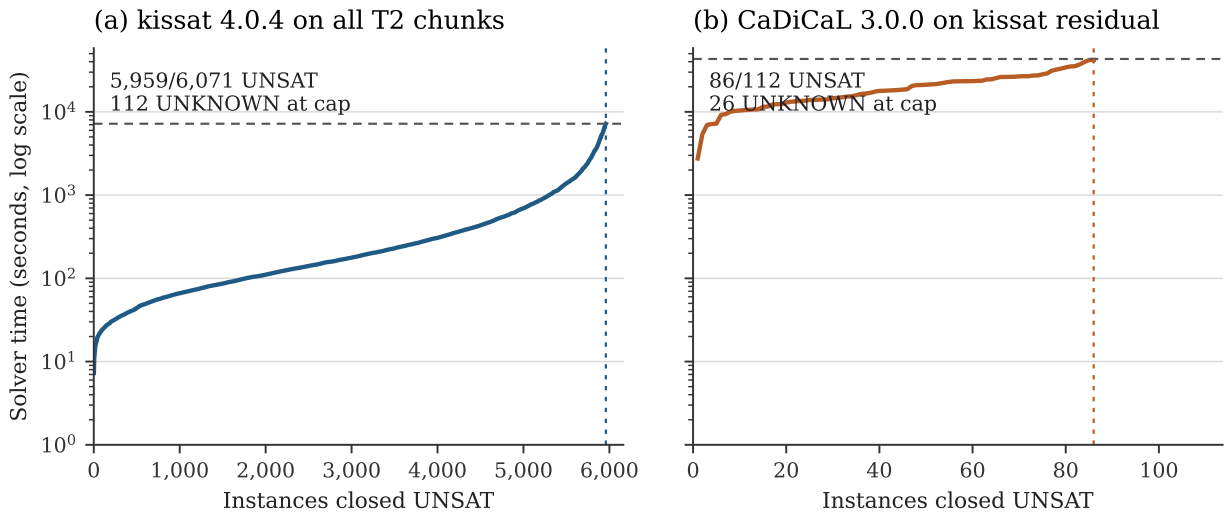

FIGURE 3. Cactus plots for the two-stage full-T2 portfolio. Curves contain only `UNSAT` rows; horizontal dashed lines mark each arm's wall cap, and vertical dashed lines mark the number closed. Panel (b) is conditioned on the 112 kissat residuals and is therefore not a common-sample solver comparison.

encoding with a 2-hour wall cap and no proof logging. Kissat closed 5,959 chunks `UNSAT` and left 112 `UNKNOWN`. We then ran native CaDiCaL 3.0.0 on precisely those 112 residual formulas with a 12-hour cap. All 112 CNF SHA-256 digests matched the corresponding kissat formulas; CaDiCaL closed 86 and left 26 `UNKNOWN`. No `SAT` row appeared.

Arm	Inputs	Cap	UNSAT	UNKNOWN	Median (s)	90th pct. (s)
kissat 4.0.4	6,071	2 h	5,959	112	180.537	1,294.351
CaDiCaL 3.0.0 on kissat residual	112	12 h	86	26	23,238.075	43,200.039
Combined portfolio	6,071	—	6,045	26	—	—

The combined portfolio therefore closes 99.57% of T2. The kissat pass consumed 3,576,567 solver-seconds (993.49 single-core hours), and the CaDiCaL residual pass consumed 2,818,578 seconds (782.94 hours). The sharp increase in residual-arm time shows why the pilot's 99% close rate did not justify a simple independent-and-identically-distributed extrapolation. Because both passes intentionally disabled proof logging, these are solver-attested closures rather than certificates; the 26-chunk residual and proof-producing reproduction of the 6,045 closures remain open targets.

3.9. **Certified exact-value regressions.** To validate the encoding and certificate pipeline on cases with known ground truth, we ran end-to-end exact-value regressions at $N = 80, 90, 100$. For each N, an independently checked witness establishes the tabulated lower bound $r_3(N)$, while a pure 3-AP/cardinality CNF asks whether a set of size $r_3(N) + 1$ exists. Kissat emitted a DRAT refutation; `drat-trim` translated it to LRAT; and the formally verified `cake_lpr` checker validated the LRAT proof [THM21]. The upper-bound certificates do not use the OEIS-derived window inequalities.

N	$r_3(N)$	Target	Vars	Clauses	Kissat (s)	DRAT–LRAT (s)	cake (s)
80	22	23	1,104	8,906	228.408	318	59
90	24	25	1,274	11,176	837.473	1,146	143

N	$r_3(N)$	Target	Vars	Clauses	Kissat (s)	DRAT–LRAT (s)	cake (s)
100	27	28	1,444	13,696	3,275.385	5,658	431

The DRAT files are 0.28, 0.74, and 1.49 GB; their LRAT translations are 1.02, 2.49, and 4.64 GB. Formula hashes match the calibration runs, lower witnesses pass the independent verifier, and all three `cake_lpr` checks report `VERIFIED UNSAT`. The checker is pinned to commit `a4323b203cc9ecd584ba7da9e3fff08135a09d5f`, with its binary digest recorded in the release manifest. These cases establish that the campaign's exporter, solver, transformation, and formally verified checking chain can recover exact values end to end; they do not certify the still-incomplete $N = 212$ sweep.

3.10. **Compute budget.** The dominant retained costs are the 100k broad expansion, the sample-500 refinement, the full-T1 long-wall HiGHS audit, and the final T2 solver portfolio. The total retained-log estimate is roughly 11,230 worker-hours; Appendix A itemizes the accounting per campaign layer. A full depth-24 sweep of the 12,582,912 AP-pruned chunks of the deployed witness-prefix lattice was outside this campaign's budget under the CP-SAT-first parameters used here. The split-policy audit reduces a complete alternative cover to 96,847 cubes, but shows that its survivors are harder at the original 60-s cap; §5.3 therefore treats proof logging and survivor conquest, not raw cube count alone, as the remaining bottleneck.

4. The structural hard pocket

The headline empirical finding of the campaign is not the absence of a 44-element 3-AP-free subset of $[1, 212]$ — which we expected from the OEIS A003002 extrapolation $r_3(212) = 43$ — but the existence, stratification, and eventual collapse of a small, robust subset of broad subproblems that resisted every solver we initially threw at them. We refer to this subset as the *hard pocket*. This section characterizes it empirically, summarizes the interventions in the order we tried them, and ends with the kissat re-attack that resolved it (§4.4) — a resolution that refutes our own working hypothesis (§4.6) and shows that the observed boundary depends on solver implementation and configuration.

4.1. **Empirical signature on the broad population.** The 100,000-chunk window-bound broad batch over chunk-id range $[11575, 111575)$ produced 93,929 `INFEASIBLE` chunks, 6,071 `UNKNOWN` chunks, and 0 `FEASIBLE` chunks under a 60-second per-chunk wall cap. The 6.07% UNKNOWN rate is non-uniform in two distinct senses.

Bucket non-uniformity. Partitioned into twenty 5,000-chunk buckets, the UNKNOWN rate varies from 1.64% to 13.60%, with the worst bucket $[61575, 66575)$ at 13.60%. The longest contiguous UNKNOWN run found in this batch was length 27 at chunk IDs 98277..98303; many other runs have length around 15. These contiguous runs are consistent with joint structural properties of the depth-24 witness-pin assignment affecting closure time; they do not by themselves separate structural effects from runtime variation.

Pin-OUT enrichment. For each of the 24 broad-split witness variables we computed the conditional UNKNOWN rate given that variable's IN/OUT assignment in the chunk. Five variables — values 68, 75, 70, 76, 91, all in the dense middle cluster $[67, 91]$ of the verified 43-witness — show a strong asymmetry: their pooled conditional UNKNOWN rate is roughly 2.4% when pinned IN and 9.7% when pinned OUT, an odds ratio of approximately 4.4. The signal is consistent across the five values. We refer to a chunk in which all five hot pins are forced OUT as a *middle-out chunk*. Middle-out chunks account for a disproportionate share of the UNKNOWN tail of the broad batch.

4.2. **The 300s-resistant tail does not collapse.** To test whether the broad-pass UNKNOWNs were merely "slow" rather than structurally hard, we ran two wall-cap recalibration experiments on a uniform random sample of 100 UNKNOWN chunks drawn from the 6,071 baseline (seed 5230). The results show a saturating, not exponential, closure curve:

Wall cap	INFEASIBLE	UNKNOWN	Close rate over baseline
60 s	0	100	0%
120 s	31	69	31%
300 s	55	45	55%

Going from 60 s to 120 s closed 31 chunks. Increasing the cap further to 300 s closed 24 more. The marginal gain per additional second decreased on this sample, and 45 chunks remained unresolved; these three points do not support a reliable extrapolation beyond 300 s.

We applied the same structural mining procedure (single-pin, pair, and triple log-odds enrichment relative to a matched `INFEASIBLE` sample) to the 45 `UNKNOWN` chunks that survived the 300s cap. The broad pin-OUT signature weakens substantially. The best high-coverage pair, $[91 = OUT, 48 = OUT]$, covers 66.67% of the survivors, but its enrichment over matched `INFEASIBLE` controls is modest. Top triples are more selective but each covers only a handful of cases. Hamming-distance clustering of survivor `fixed_in` sets shows multiple small clusters rather than a single dominant niche. We interpret this as evidence that the 300s-resistant subproblems share moderate pin-OUT structure but spread across many distinct sub-pockets of the depth-24 assignment lattice.

4.3. **HiGHS resistance and the CDCL break.** The 300s-resistant pocket might in principle be specific to the tested CP-SAT formulation and search policy. To test a substantially different solver architecture, we re-attacked all 45 survivors with the HiGHS open-source MIP solver [HH18], which combines branch-and-bound, LP relaxation, presolve, cutting planes, and primal heuristics. We used a one-hour wall cap per chunk, 8 threads per chunk, and the identical constraint set (decision-form $\sum x_i = 44$, all 11,130 3-AP triple inequalities, all 22,154 window-cardinality inequalities, plus the broad chunk's `fixed_in` and `fixed_out` assignments tightened into variable bounds).

The one-hour result was negative:

The two rows in the following table are not a common-sample comparison: the HiGHS attack targets precisely the 45 chunks that survived the 300-s CP-SAT recap, i.e. the harder subset of the 100 recap inputs.

Solver	Constraints	Wall cap	Closed (INF)	UNKNOWN	Aggregate solver wall time
CP-SAT, window-bounds	3-AP + window + symmetry	300 s	55 / 100	45 / 100	bounded by ~8.3 wall-h
HiGHS, window-bounds	3-AP + window + endpoint	3,600 s	**0 / 45**	45 / 45	45.0 wall-h

Across 162,003 solver-seconds and 3,181,316 explored MIP nodes, HiGHS did not close a single chunk. No `FEASIBLE` row appeared. Because the model was posed as a zero-objective feasibility problem with $\sum_i x_i = 44$, the recorded objective and dual bounds of 0.0 are non-diagnostic; this experiment

measures MIP closure under the tested formulation, not the quality of a maximization-model LP bound.

We then re-attacked all 45 T1 chunks under an extended 8-hour HiGHS wall, with progress logging enabled. The result is mixed: 25/45 chunks closed `INFEASIBLE`, while 20/45 returned `UNKNOWN` at the full cap. We call this 20-chunk HiGHS-resistant subset **T1b**. The audit consumed 901,073 solver-seconds and 25,196,448 MIP nodes.

We separately measured relaxation and branch-and-bound progress on all 20 T1b chunks with the optimization model $\max \sum_i x_i$, both without and with the complete window-cardinality family. These runs make the bounds meaningful: an upper bound below 44 would certify the target infeasible.

Windows	Root LP range	Incumbent range	Final UB range	Median final gap	Total nodes
off	73.0–84.5	39–40	46–57	26.92%	32,089,555
on	44.0–44.0	39–40	44–44	12.82%	6,014,774

All 40 MIP cells reached their 7,200-s cap. The windows collapse the root LP bound to the target value 44 on every chunk and reduce the explored node count by approximately 81%. They do not push any final MIP upper bound below 44, so no chunk is certified by this experiment. This resolves the ambiguity of the earlier zero-objective runs: the window inequalities are highly effective LP cuts, but on T1b they close the relaxation gap only to the decision threshold, not through it.

To test whether T1b is invariant under solver architecture more broadly, we re-attacked it with a CDCL/SAT solver (CaDiCaL via PySAT [BFFH20, IMMS18], single-threaded, 4-hour wall, encoding restricted to 3-AP triples + cardinality + chunk pins; no window-cardinality clauses). CDCL closed 18/20 chunks `UNSAT`. We call the surviving 2-chunk residual **T1c** $= \{40959, 48895\}$. A T1c diagnostic at extended wall (12 h pure CDCL) and with totalizer-encoded window constraints for lengths `{31, 100, 199}` (4 h) also returned `UNKNOWN` on both chunks. At that point in the campaign, T1c appeared resistant to every tested configuration; §4.4 reports the re-attack that overturned this.

A proof-producing rerun of the 18 CDCL-UNSAT chunks initially emitted DRAT proofs for 17 chunks; the remaining chunk, 32735, required a larger-memory proof rerun and then also emitted a DRAT proof. Independent `drat-trim` verification confirmed all 18/18 emitted proofs. The final two certificates were the slowest: 63231 verified in 49,758.11 seconds and 32735 verified in 80,960.68 seconds under the final 24-hour verifier pass [HHB13, CFHH+17].

The picture after the initial PySAT/CaDiCaL pass was configuration-specific but sharp: the tested CP-SAT and HiGHS formulations left all of T1b unresolved, while that CDCL configuration closed 18/20 with independently verified proofs. The historical residual was T1c, of size 2.

4.4. **Certified T1c closure and native CDCL same-formula comparison.** The CaDiCaL result left open whether T1c marks the edge of the tested CDCL configuration or only of one implementation. We therefore re-attacked both chunks with kissat 4.0.4 [BFFH20], single-threaded, on the same pure 3-AP/cardinality encoding (no window clauses), with DRAT logging enabled. Both chunks were refuted, and both refutations are certified end-to-end: against sha256-pinned formula files, chunk 40959 solves in 23,191 s with independent `drat-trim` verification in 41,127 s, and chunk 48895 solves in 10,083 s with verification in 16,153 s. (An initial kissat run on the same encoding had already refuted both, in 14,204.9 s and 17,621.7 s, emitting proofs of 17.4 and 14.3 GB; the certified rerun re-solved and re-verified against the same pinned formula file per chunk to establish clean certificate provenance.) The earlier CaDiCaL attempts ran through PySAT's bundled backend and default configuration. To distinguish solver identity from that invocation path,

we then ran native CaDiCaL 3.0.0 and kissat 4.0.4 on all 20 T1b chunks, using byte-identical CNFs. Both solvers closed all 20 `UNSAT`.

Solver	Closed	Median (s)	90th pct. (s)	Maximum (s)	Total (s)
CaDiCaL 3.0.0	20/20	734.0	9,700.0	28,635.8	79,421.8
kissat 4.0.4	20/20	429.0	3,467.8	13,425.5	42,782.3

All 20 paired formula hashes match. Kissat records lower wall time on 19/20 chunks, with a median paired CaDiCaL-to-kissat time ratio of 1.38. These array tasks ran opportunistically on heterogeneous Unity CPU nodes, so the ratio describes the deployed configurations rather than a processor-normalized speedup. On the two historical T1c chunks, native CaDiCaL takes 28,635.8 and 18,077.7 seconds, while kissat takes 13,425.5 and 9,963.9 seconds. Thus the original solvability separation does not survive the same-formula rerun; the first array establishes only a deployment-time difference between the tested native builds, not a processor-normalized performance separation. (A windowed kissat variant exceeded the 16 GB encode-time memory budget on both T1c chunks and produced no solver result.)

The 100-chunk pilot that motivated this comparison closed 99/100 T2 chunks with kissat. Section 3.8 replaces that estimate with the full survey: kissat closes 5,959/6,071 within two hours, and native CaDiCaL closes 86 of the 112 residuals within twelve hours. Together with the T1c certificates, these results collapse the hard pocket as a solver-independent phenomenon while preserving it as a useful graded benchmark. They also show why solver conclusions must identify the exact binary, invocation path, formula digest, and resource cap.

4.5. **Levers tested and their outcomes.** For completeness, we record every search-tuning lever we tested on the hard bucket $[61575, 66575)$ or the 300s-resistant subset. The CP-SAT/MIP-side tuning levers did not remove the hard pocket; the one qualitative exception is the CNF/CDCL switch, which closes most of T1b and therefore narrows, rather than merely tunes, the residual.

Lever	Mechanism	Result
Window-cardinality (OEIS A003002)	Add $\sum x \le r_3(L)$ for each window	UNKNOWN rate on full 10k batch: 30.33% to 1.65%. Strong but plateaus.
Split-vars reorder attempt	Intended hot-pin permutation; loader normalization made it a no-op	13.60% to 12.96% on the worst bucket is repeatability variation, not a reorder effect.
Wall-cap extension	60 s to 300 s on the broad pass	Diminishing closure gains, see §4.2.
Targeted pair-AND Tseitin propagators	Explicit `pair[a,c] = x[a]∧x[c]` BoolVars on midpoints in $[67, 91]$	Control 67.84s, treatment 71.75s on a sampled residual. No effect.
Recursive deepening	Refinement at depths 40, 48, 56	All 500 stratified-sample base chunks close, but with a non-trivial tail; the final six level-3 residuals needed up to 599.74s at the 600s cap.
HiGHS feasibility MIP	Replace CP-SAT entirely	0/45 closed at 1 hour; 25/45 closed in the full 8-hour audit, while 20/45 remained unresolved.

Lever	Mechanism	Result
HiGHS optimization MIP	Maximize $\sum_i x_i$ on T1b, windows off/on	Windows reduce nodes by 81% and tighten all root LP and final upper bounds to 44, but none below 44.
Initial PySAT/CaDiCaL CDCL	Encode T1b as CNF with 3-AP clauses + cardinality + pins, no windows	18/20 HiGHS-resistant chunks closed `UNSAT` in 4 hours; 2/20 remained `UNKNOWN`; no `SAT` rows.
Native CDCL same-formula study (§4.4)	Native CaDiCaL 3.0.0 and kissat 4.0.4 on byte-identical CNFs	Both close 20/20; kissat records lower wall time on 19/20 heterogeneous-node runs.
Full T2 portfolio (§3.8)	Kissat on 6,071 chunks, then CaDiCaL on 112 residuals	6,045/6,071 closed `UNSAT` (99.57%); 26 remain `UNKNOWN`; no proofs logged.

The pair-AND result is worth a closer note. The added constraints are mathematically redundant with the existing 3-AP triple inequalities — they encode the same forbidden configurations, just with an explicit Tseitin variable per pair. The hope was that this would give CP-SAT more "propagation hooks" in the structural pocket. The negative result is consistent with the broader picture: a local redundant encoding change is insufficient, while the solver and invocation path materially change the observed boundary.

4.6. **A refuted working hypothesis about the pocket.** During the campaign we maintained the working hypothesis that T1c marked a solver-independent hard core under practical wall caps. That hypothesis was based on repeated non-closure by CP-SAT, HiGHS, and the tested CaDiCaL setup; it was not a theorem about proof complexity or the LP relaxation. Kissat 4.0.4 refuted the hypothesis by finding multi-gigabyte DRAT proofs for both chunks, which `drat-trim` independently verified.

The native same-formula rerun narrows the correction further: T1c does not separate current native CaDiCaL and kissat by solvability, because both close 20/20 T1b formulas. It separates the initial PySAT-bundled invocation from the native runs. On the initial heterogeneous-node array, kissat records lower wall time on 19/20 byte-identical instances; this is a deployment observation, not a processor-normalized speedup. The optimization-form HiGHS study also resolves the LP question. Window bounds tighten every T1b root LP and final MIP upper bound to exactly 44, but not below it. The resulting lesson is empirical rather than universal: window cuts expose an exceptionally tight decision boundary, and implementation details determine whether CDCL crosses it within the wall cap.

4.7. **What this means for $r_3(212)$.** The combined CP-SAT, HiGHS, CaDiCaL, and kissat evidence — no `FEASIBLE` or `SAT` rows across millions of subproblems, every audited hard-pocket chunk closed `UNSAT`, and a solver portfolio closing 6,045/6,071 T2 chunks — strongly supports the OEIS-conjectured value $r_3(212) = 43$, and the kissat results substantially change the outlook on proving it. The lower bound $r_3(212) \geq 43$ from the verified witness in §2 is unaffected; the trivial upper bound $r_3(212) \leq 44$ follows from monotonicity and OEIS $r_3(211) = 43$, so the gap between our certified bounds is exactly one.

What blocks the upper bound is complete coverage and certification of the remaining search space. The deployed witness-prefix cover contains 12,582,912 chunks, of which this campaign processed the 100,000-chunk expansion plus samples. The split-policy audit supplies an alternative, equally complete global-degree cover with 96,847 survivors, but its sampled survivors are individually harder at the 60-s CP-SAT cap. The full T2 survey leaves 26/6,071 chunks unresolved after the

two-solver portfolio, and its 1,776.43 single-core hours measure the heavy tail directly. A certified sweep would be of the same general character as the Pythagorean-triples and Schur-five campaigns [HKM16, Heu18], with a deterministic split generator providing the cube decomposition. We release the benchmark instances and generator in §5 so that this sweep, or a sharper per-chunk method, can be attempted directly.

5. Open problem and benchmark release

The campaign establishes strong empirical support for $r_3(212) = 43$ (§3.5, §4.4). Native CaDiCaL and kissat both close every audited T1b instance, and their combined full-T2 portfolio closes 99.57% of the 6,071 broad residuals, while complete coverage and certification of T3 remain open. This section states the residual problem cleanly and releases the instances as a tiered benchmark — both as a concrete path toward R3-212-UB and as a solver-grading ladder for the tested CP-SAT, HiGHS, and CDCL configurations.

5.1. Statement of the open problem.

> **Problem (R3-212-UB).** Let $A \subseteq [1, 212]$ with $|A| = 44$. Determine whether there exists A containing no nontrivial 3-term arithmetic progression. The verified 43-element witness in `results/N212_K43_witness.json` (§2.1) and the value $r_3(211) = 43$ from OEIS A003002 jointly imply that any such A must contain both endpoints 1 and 212. Equivalently, either exhibit one such A or certify the infeasibility of the decision problem $\sum_i x_i = 44$ over the 3-AP constraint family on $[1, 212]$ with $x_1 = x_{212} = 1$.

A solution to R3-212-UB resolves the unit gap $r_3(212) \in \{43, 44\}$ in either direction. Our multi-million-subproblem CP-SAT campaign, the 45-instance HiGHS attack, and the CaDiCaL/kissat re-attacks returned 0 `FEASIBLE`/`SAT` rows, which is evidence for the infeasibility branch but not a proof. The benchmark instances released below pinpoint the subproblems on which solver technologies diverge.

5.2. Benchmark instance tiers.

We release five benchmark tiers. Each JSONL row records the chunk ID, fixed assignments, split-policy metadata, solver status, and timing data; the experiment-specific constraint family is reconstructed by the repository scripts. The tiers form a ladder from the smallest solver-resistant pocket to the full upper-bound proof.

Tier	Artifact	Size	Resistance level	Recommended target
T1a	T1a JSONL	25 chunks	closed by HiGHS at 8h	reference / regression test
T1b \ T1c	T1b-minus-T1c JSONL	18 chunks	HiGHS-resistant; closed by CDCL; all 18/18 emitted DRAT proofs verified by `drat-trim`	certified CDCL benchmark
T1c	T1c JSONL	2 chunks	resisted the initial PySAT/CaDiCaL path; both native CaDiCaL and kissat close them; kissat DRAT-certified	native-CDCL performance calibration
T2	T2 JSONL	6,071 chunks	survived 60-s CP-SAT; 6,045 closed by the native CDCL portfolio, 26 unresolved	certified closure of the expansion residual
T3	deterministic generators	12,582,912 witness-prefix or 96,847 global-degree cubes	complete covers; global-degree survivors are harder at 60-s CP-SAT	full upper-bound proof

The concrete filenames are `results/N212_K44_t1a25.jsonl`, `results/N212_K44_t1b_minus_t1c.jsonl`, `results/N212_K44_t1c2.jsonl`, and `results/N212_K44_window100k_unknowns.jsonl`. The global-degree survivor sample and exact-count metadata are released as `results/global_degree_survivor_sample100.jsonl` and `results/global_degree_survivor_sample100_summary.json`.

Tier T1c is the benchmark's compact native-CDCL calibration point. The two chunks defeat the initial PySAT/CaDiCaL path, but native CaDiCaL 3.0.0 and kissat 4.0.4 both close them on byte-identical formulas; kissat records lower wall time on both and supplies end-to-end certified DRAT proofs (§4.4). Lower solve or verification cost than the reported runs is a directly comparable benchmark improvement.

Tier T2 is the canonical "close the campaign at the broad layer" target: closing all 6,071 chunks of the 100k expansion batch is necessary, though not sufficient, for a full upper-bound proof. The completed no-proof survey uses 993.49 single-core hours for kissat on all 6,071 chunks and another 782.94 hours for native CaDiCaL on the 112 kissat residuals. The portfolio closes 6,045 and leaves 26 unresolved. Those 26 formulas, together with a proof-producing reproduction of the 6,045 solver-attested closures, are the next concrete certification targets.

Tier T3 is the full upper-bound certificate: closure of any complete depth-24 AP-pruned cover suffices. The historical witness-prefix generator emits 12,582,912 survivors; the audited global-degree generator emits 96,847. Both covers are deterministic and sound because every omitted raw assignment already contains a pinned 3-AP. The smaller cover is not automatically easier: the matched ablation's three sampled global-degree survivors all time out at 60 s, so conquer cost must be measured separately from cube count.

5.3. **Minimum-viable proof requirements.** A formal proof of $r_3(212) \leq 43$ from the released benchmark requires:

1. **Closure of tier T3.** Every chunk in one complete depth-24 AP-pruned cover must return `INFEASIBLE` under a verified solver, or one chunk must return a `FEASIBLE` 44-element witness.
2. **Solver verification.** Every infeasibility result used in the proof must be justified by a machine-checkable certificate (DRAT, LRAT, or an equivalent proof format) checked by a trusted or formally verified checker. Agreement between two unverified solvers is useful validation but is not a formal certificate. Section 6.2 records the present gaps.
3. **Constraint-set verification.** The 11,130 3-AP triple inequalities, the 22,154 window-cardinality inequalities, and the endpoint forcing must be checked against the formal definition of $r_3(N)$. The verifier `r3_verify.py` performs this check on any candidate witness. A solver-independent, standard-library audit regenerates the high-level constraints, checks monotonic unit increments in the source b-file, reproduces both counts, and records SHA-256 digests for the b-file, AP family, window family, and complete high-level model; its JSON report is included in the artifact release.

The minimum-viable result short of a full proof is the certified closure of tier T2: verified proofs for all 6,071 broad-residual chunks. The completed survey measures 1,776.43 single-core hours of no-proof solving and leaves 26 chunks unresolved. Proof emission and verification can dominate solving — as the multi-gigabyte T1c certificates demonstrate — so this observed solver cost is a lower planning baseline, not a forecast for certified T2 or T3. The optimization-form MIP experiment of §4.3 supplies a complementary negative result: window cuts tighten every T1b upper bound to 44, but never below the decision threshold.

5.4. **Follow-on approaches.** The following techniques remain natural follow-on directions after the completed solver portfolio and optimization-form MIP study:

- **Fourier-analytic upper bounds.** Behrend-style constructions achieve $r_3(N) = N \cdot exp(-c \cdot sqrt(logN))$ lower bounds; a matching Fourier-analytic upper bound for small N would give a certificate independent of combinatorial search. The Bloom–Sisask framework for $r_3(N) = O(N/(logN)^{1+c})$ is asymptotic but the underlying density-increment machinery may yield finite-N bounds tighter than the OEIS window-cardinality family.
- **Multi-window partition bounds.** OEIS A003002 is used here as a single-window family $\sum_{i=a}^{a+L-1} x_i \leq r_3(L)$. A partition-cardinality bound across multiple disjoint windows of varying length might cut the LP relaxation more aggressively than any single-window family.
- **SAT with proof logging at scale.** A pure CNF encoding closed all 20 T1b chunks under both native CaDiCaL and kissat; all 18 proofs from the initial PySAT/CaDiCaL closures and both kissat T1c proofs are independently verified. The full no-proof T2 portfolio closes 6,045/6,071. The research target has therefore moved from single instances to the certified sweep: proof-producing kissat over all of T2 and ultimately T3, with verification and archival at Pythagorean-triples scale [HKM16]. Open per-instance questions remain at the margins — especially the 26 full-survey residuals and whether alternative cardinality encodings shrink the multi-GB certificates of the hardest chunks.
- **Lean/formal-proof-search benchmark.** The repository now includes a compact Lean 4 / Mathlib 4 formalization [dMU21, The20] under `lean/`: shared definitions (`R3Base.lean`), the verified 43-witness statement (`R3_212_Witness.lean`), and two AlphaProof-style T1c targets (`R3_T1c_40959.lean`, `R3_T1c_48895.lean`) with the expected answer `false`. These files are intended as a starting point for an A003002 / $r_3(N)$ entry in formal-conjecture repositories [Goo26]. Recent AlphaProof Nexus-style workflows have resolved Erdős problems of comparable complexity at low per-problem cost when a Lean target exists [TKS+26], so T1c is well-shaped for that workflow.
- **Custom branch-and-bound.** A solver that exploits problem- specific symmetries and dominance relations — for instance, reflection symmetry after endpoint forcing, or domination of one window-bound by another at specific fixed-pin configurations — could prune branches that neither CP-SAT nor HiGHS recognize as redundant.
- **Symmetric Difference Encoding / Lasserre hierarchy at low degree.** A degree-4 or degree-6 SDP relaxation of the 3-AP-free constraint might separate the surviving T1 instances from feasibility where the LP relaxation cannot.

We do not advocate for any single direction. The benchmark release is intended to let specialists pick the technique closest to their toolkit.

5.5. **Reproducibility and release.** The campaign code, configuration, and result logs are released at the repository accompanying this preprint. A claim-to-artifact map and the shortest verification commands are collected in `ARTIFACT_REPRODUCIBILITY.md`. Specifically:

- All Python sources implementing the architecture of §2 are present with module-level documentation.
- The SLURM scripts (`submit_*.sbatch`) and the driver `unity_handoff.sh` reproduce the full campaign on any SLURM-compatible cluster with OR-Tools and `highspy` installed.
- The verified 43-witness, the OEIS A003002 b-file used for window bounds, the broad-pass result logs, the recap residuals, the HiGHS attack logs, the T1a/T1b/T1c benchmark JSONLs, the full T2 solver survey, the native-CDCL same-formula and optimization-MIP comparisons, the exact-value certificates at $N = 80, 90, 100$, the Lean T1c proof-search targets, the independent high-level model audit and SHA-256 report, environment/version captures for the main solver stack, and the scripts needed to generate T3 are included in the repository or its associated artifact archive.

- Existing single-instance entry points (`r3_split_cpsat.py` and `r3_highs_attack.py`, and `r3_sat_attack.py`) can rerun any row of T1c, T1b, or T2 by chunk ID.

The large solver artifacts are archived on Zenodo at https://doi.org/10.5281/zenodo.20463334. Version 1 contains the CNF/DRAT proof artifacts for the 18 CDCL-closed T1b \ T1c chunks, the available LRAT outputs, verification summaries, SLURM logs, solver outputs, SHA256 manifests, and reconstruction instructions for split archives [Erg26]. The journal-release version adds the certified T1c formula/proof pairs and provenance records, the full T2 survey, the same-formula solver and optimization-MIP outputs, and the exact-value DRAT/LRAT certificate suite. Each formula and proof is accompanied by a SHA-256 digest and a machine-readable provenance record.

We welcome correspondence on partial results: any solver run that closes a strict subset of T1 or T2 is a meaningful incremental contribution, even if the full upper bound remains open.

6. Discussion

We close with three observations: what the architecture transfers to, where it breaks down, and what the campaign suggests about the broader proof-search problem class for r_3 upper bounds.

6.1. **Reusability for adjacent N.** The five-component architecture of §2 is parametric in N and K and depends on two external inputs: a verified `(K - 1)`-element lower-bound witness for $r_3(N)$ and the OEIS A003002 b-file prefix for window-cardinality pruning at lengths $L \le N-1$. Both inputs are standard finite data at the current frontier: exact values through $N = 211$ are tabulated by Cariboni/OEIS A003002, and any new run also requires an explicit verified lower-bound witness at the target size. We expect the architecture to apply directly to $N \in \{213, \dots, 220\}$ once such witnesses are supplied, with three caveats:

1. The depth of the broad split ($d = 24$ for $N = 212$) is empirical and likely needs to grow with N. The number of feasible-after-pruning chunks at depth d scales roughly geometrically with the size of the witness, so the broad layer for larger N will emit more chunks, and per-chunk cost will also rise as the number of variables and 3-AP triples grows.
2. The density $r_3(N)/N$ decreases slowly while $r_3(N)$ itself is nondecreasing. At successive frontier decisions, both the number of variables and the target cardinality generally grow, so we expect the `UNKNOWN` rate at a fixed wall cap to increase, although this extrapolation has not been tested beyond $N = 212$.
3. The endpoint-forcing argument generalizes: if $r_3(N-1) = K-1$, then any K-element 3-AP-free subset of $[1, N]$ contains both endpoints. This is an N-specific logical reduction; the verified witness and the tabulated window bounds are the other problem-specific inputs.

Within the OEIS A003002 frontier, the architecture is therefore a drop-in tool for any incremental $r_3(N')$ upper-bound attempt, with the same caveat the present campaign documents: the hard pocket of §4 will likely have an analogue at larger N, possibly more severe — and whether that analogue again falls to the strongest available CDCL implementation, as it did at $N = 212$, is an open empirical question.

6.2. **Limitations.** The campaign's main limitations are certificate coverage and scale: most closures are solver-attested rather than third-party-verified, and the processed chunk ranges cover a fraction of the full depth-24 sweep. Section 5 prices the remaining path; here we record the narrower limitations of the present implementation:

- **Machine-checkable certificates cover every CDCL-resolved hard-pocket chunk and three exact regressions, but not the full campaign.** Follow-up proof-producing runs emitted DRAT proofs for all 18 CaDiCaL-resolved T1b \ T1c chunks, and independent `drat-trim` verification confirmed all 18/18, and the two kissat T1c proofs are likewise

verified end-to-end against sha256-pinned formulas (§4.4). In addition, the exact values at $N = 80, 90, 100$ are established end to end by independently verified lower witnesses and `cake_lpr`-checked LRAT upper-bound proofs (§3.9). The 6,045 full-T2 portfolio closures ran without proof logging by design; the 25 HiGHS-closed T1a chunks and the ~93,929 broad-pass CP-SAT INFEASIBLE returns remain solver-attested rather than third-party-verified. Closing the CP-SAT/MIP side of this gap would require either a SAT re-encoding of each chunk or a verified LP-duality-style certificate format we are not aware of in the open MIP ecosystem.

- **Compute coverage is incomplete.** The campaign processed 100,000 of the 12,582,912 AP-pruned chunks in the historical witness-prefix cover, plus refinements and a complete no-proof solver survey of all 6,071 T2 residuals. The audited global-degree policy yields a complete 96,847-cube cover, but only a targeted survivor sample has been run so far. A full certified sweep under a CDCL-first strategy is the natural successor project rather than a rerun of the historical cover.
- **Most timing studies use heterogeneous cluster nodes.** Unity's shared CPU partition includes multiple AMD and Intel processor generations. Formula-level status and certificate conclusions are invariant to node placement, but unnormalized wall-time ratios should be read as deployment measurements rather than hardware-isolated solver speedups.
- **Witness dependence is structural, not adversarial.** The broad split is anchored to one specific 43-witness. A different witness would induce a different chunk-ID space and possibly a different hard pocket. We did not test whether the hard pocket is witness-invariant; if it is not, alternative witnesses might yield a lower-cost partition of the same decision problem.
- **The Lean benchmark is not yet upstreamed.** We did not find a public A003002 / $r_3(N)$ entry in `google-deepmind/formal-conjectures` as of the campaign date. The repository includes a Lean 4 formalization of the witness and T1c targets, but upstreaming and independent typechecking in that ecosystem remain follow-on work.

The third limitation suggests a useful follow-on: repeat the complete cube decomposition with one or more structurally distinct 43-witnesses and compare the resulting hardness distributions in the common $[1, 212]$ coordinate system. Each witness induces a different partition of the same search space; the partitions should be compared or combined adaptively, not treated as a disjoint union.

6.3. **What the campaign suggests about r_3 upper-bound search.** The dominant finding is configuration-level rather than paradigm-level. The tested CP-SAT and HiGHS feasibility formulations leave the same 20 T1b chunks unresolved. The initial PySAT/CaDiCaL path closes 18/20, whereas native CaDiCaL 3.0.0 and kissat 4.0.4 both close 20/20 on byte-identical formulas. Kissat records lower wall time on 19/20, but the solvability gap disappears. The optimization-form MIP experiment adds a complementary formulation result: window cuts reduce node count by about 81% and tighten every upper bound to 44, yet none crosses below the target.

Two narrower suggestions follow from the lever inventory of §4.5. First, the window-cardinality family from OEIS A003002 produced the largest measured CP-SAT improvement: an approximately 28-percentage-point reduction on controlled broad batches. The intended variable-reordering A/B was an implementation no-op and supports no ordering conclusion; the single pair-AND trial was inconclusive. Longer wall caps did close 55/100 sampled broad residuals by 300 seconds, but left a substantial tail. These results rank the tested interventions; they do not rule out untested formulations or search policies.

The CDCL experiments (§4.3–§4.4) close all 20 audited T1b chunks under both native solvers. Extending the comparison to the full T2 tier produces a two-stage portfolio result: kissat closes 5,959/6,071 within two hours and CaDiCaL closes 86 of the 112 residuals within twelve hours. The

remaining 26 chunks define a substantially sharper practical target than the earlier 6,071-chunk residual.

Combined, these suggest three immediate follow-ups: proof-producing closure of the 26 T2 residuals, proof-producing reproduction of the 6,045 solver-attested T2 closures, and a controlled comparison of decomposition policies at fixed cube count and solver budget. The benchmark release of §5 is built to support these tests.

The campaign's working hypothesis (§4.6) — that T1c was resistant to current complete solvers under practical wall caps — was refuted by the kissat runs. The broader lesson is methodological: archive the unresolved instances and exact formulas, and state resistance claims at the level of tested configurations so that later solvers can reproduce or overturn them.

Appendix A. Compute accounting

The retained logs provide measured solver wall time for the main diagnostics. Since CP-SAT tasks used 8 workers, the worker-hour estimates below multiply recorded solver-wall seconds by 8; for HiGHS, the analogous estimate is 8 threads per one-hour task. These are retained-log estimates, not exact SLURM accounting totals.

Layer	Worker-hours	Notes
Retained broad logs	~2,276	includes the 10k no-window run, the 10k window run, and the $[575, 675)$ A/B
Monolithic baselines	~ 816	six 24-h cells: two CP-SAT at 8 workers, two HiGHS at 8 threads, and two single-core CDCL runs
Sample-100 refinement	~375	depth-40 plus one depth-48 tail
Sample-500 refinement	~1,978	depth-40, depth-48, depth-56, and depth-64 cleanup
Recap and worst-bucket A/Bs	~598	reorder, 300s walltime, 120s recap, 300s recap
HiGHS attack on 45 survivors	~360	1-h wall, 8 threads per task
Full T1 HiGHS long-wall audit	~2,002	45 T1 chunks, 8-h cap, job 58782313; estimated from retained `seconds` fields
T1b CDCL first run	~19.8	20 single-core tasks, retained JSON row time 71,216.85 seconds
T1b proof-producing rerun + T1c diagnostic	~120	proof emission + 4-cell T1c grid
drat-trim verification	~ 61	jobs 58952708, 59058393, and 59383874; all 18 CDCL-resolved T1b $\setminus$ T1c chunks VERIFIED
Kissat T1c initial runs	~ 8.8	two single-core solves, 31,826.6 retained solver-seconds
Kissat T2 pilot	~ 9.7	100 single-core tasks, 34,819.8 retained solver-seconds
Certified T1c rerun	~ 25.2	two single-core solves plus two independent proof-verification runs

Layer	Worker-hours	Notes
Native CDCL same-formula comparison	~ 34.0	CaDiCaL 3.0.0 and kissat 4.0.4 on 20 byte-identical T1b formulas
Optimization-form T1b MIP	~ 640	20 chunks with windows off/on; retained 2-hour MIP-cap runs
Full T2 kissat survey	~ 993.5	6,071 single-core tasks; 3,576,567 retained solver-seconds
T2 CaDiCaL residual pass	~ 782.9	112 single-core tasks; 2,818,578 retained solver-seconds
Exact-value calibration and certification	~ 8	$N = 80, 90, 100$ solves, DRAT-to-LRAT conversion, and `cake_lpr` checks
Matched split-policy ablation	~ 55.6	500 raw assignments, five sequential policies, eight-worker CP-SAT; 25,042 retained solver-seconds
Total retained logs	$\sim 11{,}230$	excludes interactive debugging, failed encode attempts, and logs not retained locally

The campaign fits within a single academic SLURM allocation on a CPU partition. SLURM job identifiers are retained in the table and in the released logs so that every worker-hour estimate can be traced to a specific recorded run.

Acknowledgments

This work was performed on the Unity High Performance Computing (HPC) platform, a collaborative, multi-institutional resource supported by the Massachusetts Green High Performance Computing Center (MGHPCC) and its member institutions. We acknowledge OEIS contributor Cariboni for the A003002 b-file at the current frontier $N \leq 211$.

Data and code availability

Source code, benchmark generators, compact result tables, and manuscript sources are available at https://github.com/memoatwit/erdos_1194. Versioned solver artifacts, including formulas, proof objects, hashes, and provenance records, are archived at https://doi.org/10.5281/zenodo.20463334.

Author contributions

M.E. conceived the study, designed and executed the computational campaign, verified the results, developed the released software and benchmarks, and wrote the manuscript.

Competing interests

The author declares no competing interests.

Use of generative AI

In the preparation of this work, the author used Anthropic Claude (Claude Sonnet 4.5 and Claude Opus 4.6, accessed via the Claude Code agent interface) and OpenAI Codex (GPT-5, accessed via the Codex agent interface) for drafting and revision assistance, code generation and review, experiment planning, repository scaffolding, and computational orchestration. The author made

the final scientific and experimental decisions, reviewed the generated text and code, independently checked the reported numerical and mathematical claims, and takes full responsibility for the content. The tools do not appear as authors.

References

[Beh46] F. A. Behrend, *On sets of integers which contain no three terms in arithmetical progression*, Proceedings of the National Academy of Sciences **32** (1946), no. 12, 331–332.

[BFFH20] Armin Biere, Katalin Fazekas, Mathias Fleury, and Maximilian Heisinger, *CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT competition 2020*, Proceedings of SAT Competition 2020: Solver and Benchmark Descriptions, Department of Computer Science Report Series B, University of Helsinki, 2020, pp. 51–53.

[BHMN22] Joshua Brakensiek, Marijn J. H. Heule, John Mackey, and David Narváez, *The resolution of Keller's conjecture*, Journal of Automated Reasoning **66** (2022), 277–300.

[BS20] Thomas F. Bloom and Olof Sisask, *Breaking the logarithmic barrier in Roth's theorem on arithmetic progressions*, arXiv preprint (2020), arXiv:2007.03528.

[Car24] Lorenzo Cariboni, *b-file for OEIS A003002 up to $n = 211$*, OEIS A003002 b-file, 2024.

[CFHH+17] Luís Cruz-Filipe, Marijn J. H. Heule, Warren A. Hunt, Jr., Matt Kaufmann, and Peter Schneider-Kamp, *Efficient certified RAT verification*, CADE-26 (2017), 220–236.

[dMU21] Leonardo de Moura and Sebastian Ullrich, *The Lean 4 theorem prover and programming language*, CADE-28, 2021, pp. 625–635.

[Dyb12] Janusz Dybizbański, *Sequences containing no 3-term arithmetic progressions*, Electronic Journal of Combinatorics **19** (2012), no. 2, #P15.

[Erg26] Mehmet Ergezer, *Artifacts for the $r_3(212)$ witness-split computational campaign*, Zenodo dataset, 2026, https://doi.org/10.5281/zenodo.20463334.

[GHG+21] Ambros Gleixner, Gregor Hendel, Gerald Gamrath, Tobias Achterberg, Michael Bastubbe, Timo Berthold, Philipp Christophel, Kati Jarck, Thorsten Koch, Jeff Linderoth, Marco Lübbecke, Hans D. Mittelmann, Derya Ozyurt, Ted K. Ralphs, Domenico Salvagnin, and Yuji Shinano, *MIPLIB 2017: Data-driven compilation of the 6th mixed-integer programming library*, Mathematical Programming Computation **13** (2021), 443–490.

[GMN22] Stephan Gocht, Ciaran McCreesh, and Jakob Nordström, *An auditable constraint programming solver*, 28th International Conference on Principles and Practice of Constraint Programming (CP 2022), LIPIcs, vol. 235, Schloss Dagstuhl, 2022, pp. 25:1–25:18.

[Goo26] Google DeepMind, *formal-conjectures: A lean library of open mathematical conjectures*, 2026, Accessed 2026-05-25.

[GS04] Carla P. Gomes and Meinolf Sellmann, *Streamlined constraint reasoning*, Principles and Practice of Constraint Programming – CP 2004, Lecture Notes in Computer Science, vol. 3258, Springer, 2004, pp. 274–289.

[GW99] Ian P. Gent and Toby Walsh, *CSPLib: A benchmark library for constraints*, Principles and Practice of Constraint Programming – CP 1999, Lecture Notes in Computer Science, vol. 1713, Springer, 1999, pp. 480–481.

[Heu18] Marijn J. H. Heule, *Schur number five*, Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18), 2018, pp. 6598–6606.

[HH18] Q. Huangfu and J. A. J. Hall, *Parallelizing the dual revised simplex method*, Mathematical Programming Computation **10** (2018), no. 1, 119–142, HiGHS solver; https://highs.dev.

[HHB13] Marijn J. H. Heule, Warren A. Hunt, Jr., and Armin Biere, *Trimming while checking clausal proofs*, FMCAD 2013, IEEE, 2013, pp. 181–188.

[HKM16] Marijn J. H. Heule, Oliver Kullmann, and Victor W. Marek, *Solving and verifying the boolean Pythagorean triples problem via cube-and-conquer*, Theory and Applications of Satisfiability Testing – SAT 2016, Lecture Notes in Computer Science, vol. 9710, Springer, 2016, pp. 228–245.

[HKWB12] Marijn J. H. Heule, Oliver Kullmann, Siert Wieringa, and Armin Biere, *Cube and conquer: Guiding CDCL SAT solvers by lookaheads*, Hardware and Software: Verification and Testing (HVC 2011), Lecture Notes in Computer Science, vol. 7261, Springer, 2012, pp. 50–65.

[IMMS18] Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva, *PySAT: A Python toolkit for prototyping with SAT oracles*, SAT 2018, 2018, pp. 428–437.

[KM23] Zander Kelley and Raghu Meka, *Strong bounds for 3-progressions*, arXiv preprint (2023), arXiv:2302.05537.

[OEI24] OEIS Foundation Inc., *The on-line encyclopedia of integer sequences, sequence A003002: Salem–spencer numbers*, 2024, b-file extended by L. Cariboni; accessed 2026.

[PF24] Laurent Perron and Vincent Furnon, *OR-Tools, version 9.x*, 2024.

[Rot53] K. F. Roth, *On certain sets of integers*, Journal of the London Mathematical Society **28** (1953), 104–109.
[SS42] R. Salem and D. C. Spencer, *On sets of integers which contain no three terms in arithmetical progression*, Proceedings of the National Academy of Sciences **28** (1942), no. 12, 561–563.
[The20] The Mathlib Community, *The Lean mathematical library*, CPP 2020, 2020, pp. 367–381.
[THM21] Yong Kiam Tan, Marijn J. H. Heule, and Magnus O. Myreen, *cake_lpr: Verified propagation redundancy checking in CakeML*, Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2021), Lecture Notes in Computer Science, vol. 12652, Springer, 2021, pp. 223–241.
[TKS+26] George Tsoukalas, Anton Kovsharov, Sergey Shirobokov, Anja Surina, Moritz Firsching, Gergely Bérczi, Francisco J. R. Ruiz, Arun Suggala, Adam Zsolt Wagner, Eric Wieser, Lei Yu, Aja Huang, Miklós Z. Horváth, Andrew Ferrauiolo, Henryk Michalewski, Codrut Grosu, Thomas Hubert, Matej Balog, Pushmeet Kohli, and Swarat Chaudhuri, *Advancing mathematics research with AI-driven formal proof search*, arXiv preprint (2026), arXiv:2605.22763.

Wentworth Institute of Technology, Boston, MA, USA

Email address: ergezerm@wit.edu